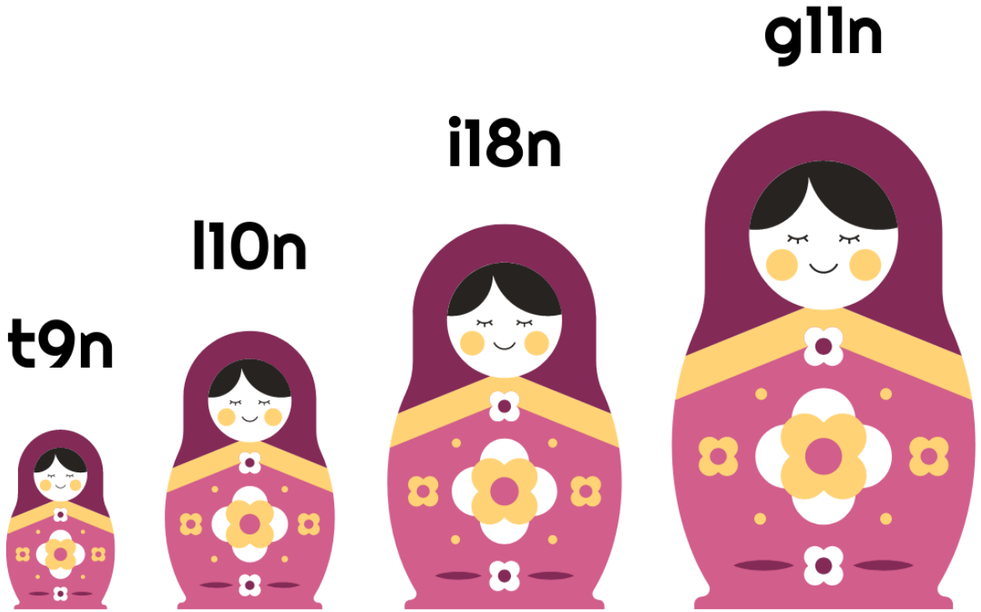
Global Software Development: Introduction to I18N and L10N

Creating software for global users requires specialized knowledge of internationalization and localization practices.

Siva R Jasthi,
Department of Computer Science and Cybersecurity
Metropolitan State University

by Siva Jasthi



https://docs.tolgee.io/blog/localization-basics-S01E01

Globalization Overview

- Diverse markets
- Cultural adaptation
- Technical standardization
- Universal accessibility

Critical Technical Considerations

Character vs Logical Character

Single byte vs multi-byte strings

API Design

Handle strings, not individual bytes

File Encoding

UTF-8 for all files

Externalizing Strings

Never hardcode Strings

INTERNATIONALIZATION

Lamguage Confonrive entemuatian)
bicjzans 202408
Byctra: #/chanauidgersters.

Software (HMKaIWihernalizure)
Noltain ; Up)
puttain, ZUMMH-(FOHATW/T);70X), G4P
#RPN4/azaz ; S)
#aeas: 'tut, a newx)
chacse: (Ake. FLLZ, FHPMS (AR)
#"luanagan, typl (ing
#7 Viulal is.-pau,)
entersfyzals (kite,
ateriguta panglor there
#im: sullal hera, munil;
rethantion: rextral, istaltnangita.

Lundestiza: rewn language
Bulla: Packrang, lmlg
Kilnn: (550, Mun, PHDT
Viltas: Strase, lyg)
Bizuba: Strkang (inlg
Yichansita, dfituurue

Encodng: Confoding

Vidras: fart intesting software,
Week inan: 780,PH), FVCCE7O/S).
#illuaugaa ; reaman,cholint enutlin,
fuclectial vmdla; iansture

Course Outline

Globalization

Overall process for worldwide deployment

Internationalization

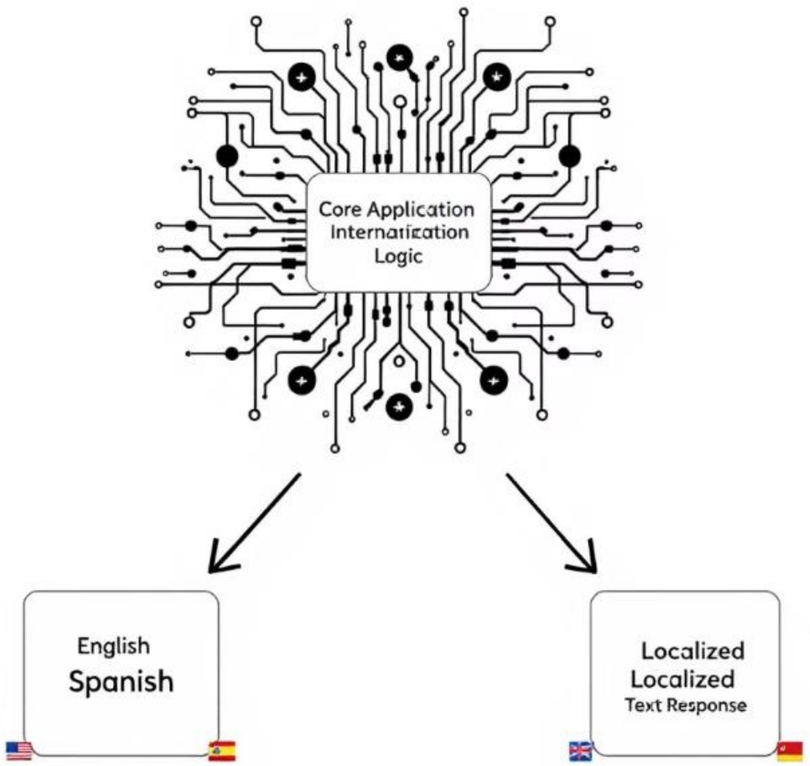
Enabling software to be localized

Localization

Adapting to specific regions

Technical Foundation

Bits, bytes, history, and Unicode solutions



Internationalization (i18n)

1. **Definition**

   Enabling software for multiple locales

2. **Notation**

   i + 18 letters + n = i18n

3. **Proactive Approach**

   Not the same as retrofitting

4. **Design Principle**

   Plan for global use from start

Localization (L10N)

Adapt to Language

Translate text elements

Regional Settings

Adjust to local conventions

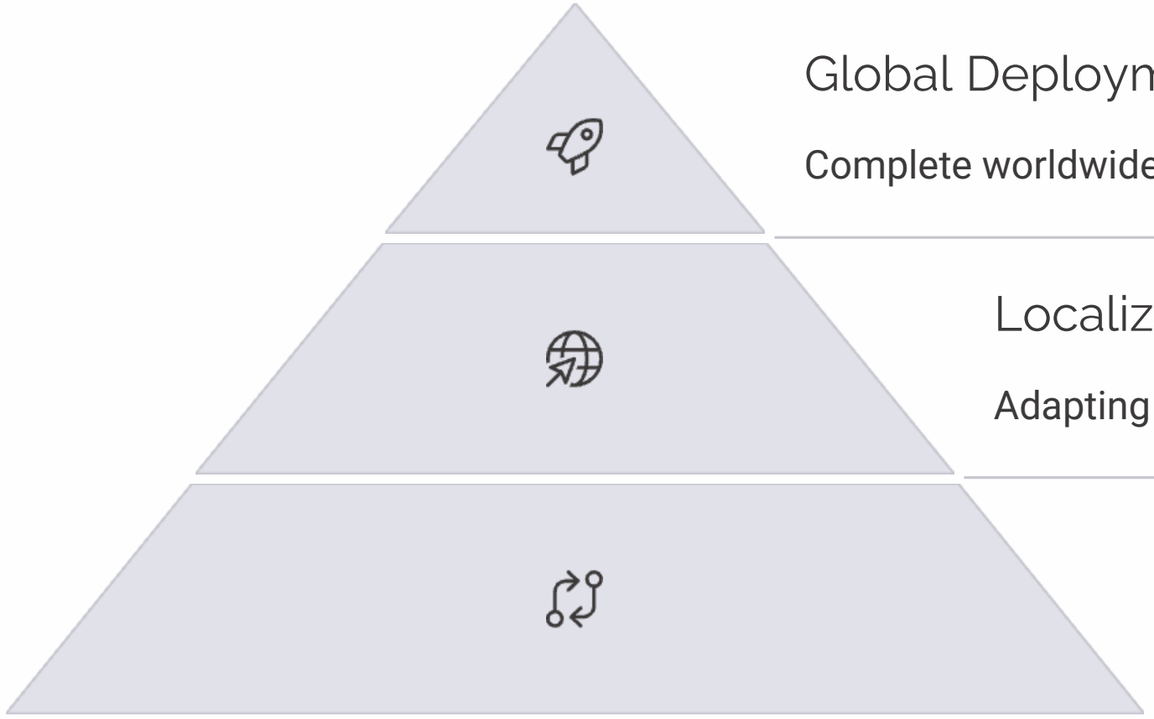
Quality Assurance

Test in target environment

Cultural Adaptation

Modify imagery and symbols

Globalization (g11n)



Global Deployment

Complete worldwide release

Localization (L10N)

Adapting for specific markets

Internationalization (i18n)

Building adaptable foundation

Translation Process

Extract Content

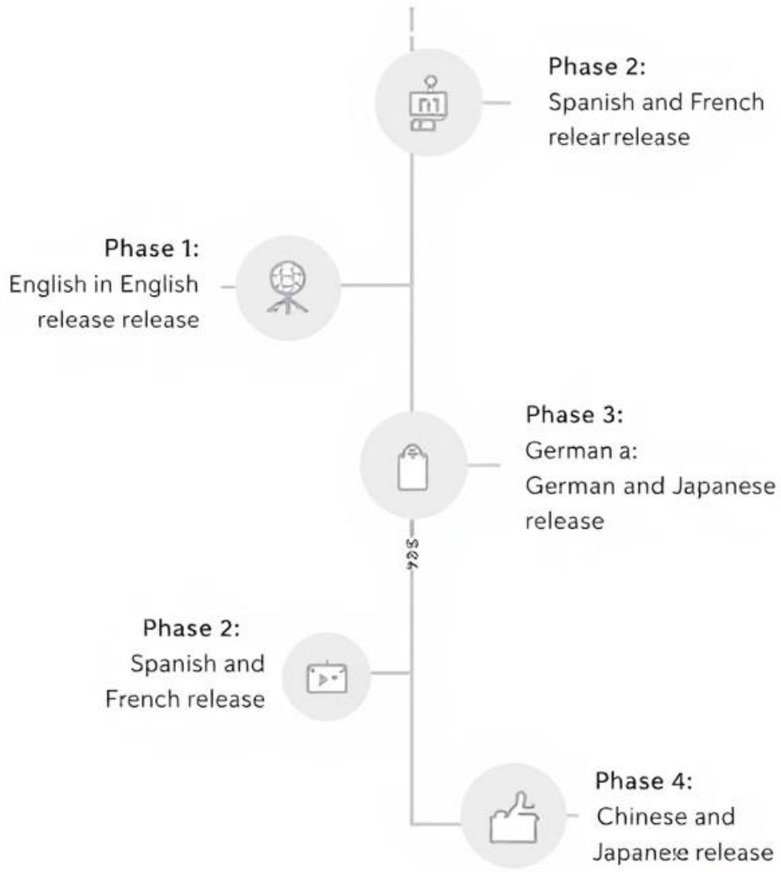
Identify translatable strings

Cultural Adaptation

Context-aware not literal translation

Localized Examples

"Starbucks" → "Coffee House"



New software releases soft cline

Expoy to love that you macreadurace and louking by the endl a Chinese arith un, and claving 22, 2013.

Translation Models

1. **Sequential Model**

   English first, localized versions later

2. **Parallel Model**

   Freeze strings 3 months before release

3. **Simultaneous Release**

   All languages launch together

Transliteration

Definition

Converting text between scripts

Preserves pronunciation, not meaning

Examples

Latin → Cyrillic

"New York" → "Нью-Йорк"

Tools

Google Translate

Specialized conversion systems

GILT Framework

Globalization (g11n)

Comprehensive global strategy

Internationalization (i18n)

Technical enablement

Localization (L10N)

Regional adaptation

Translation (t9n)

Content conversion

Characteristics of I18N Applications

Externalized Text

UI labels stored outside code

Dynamic Loading

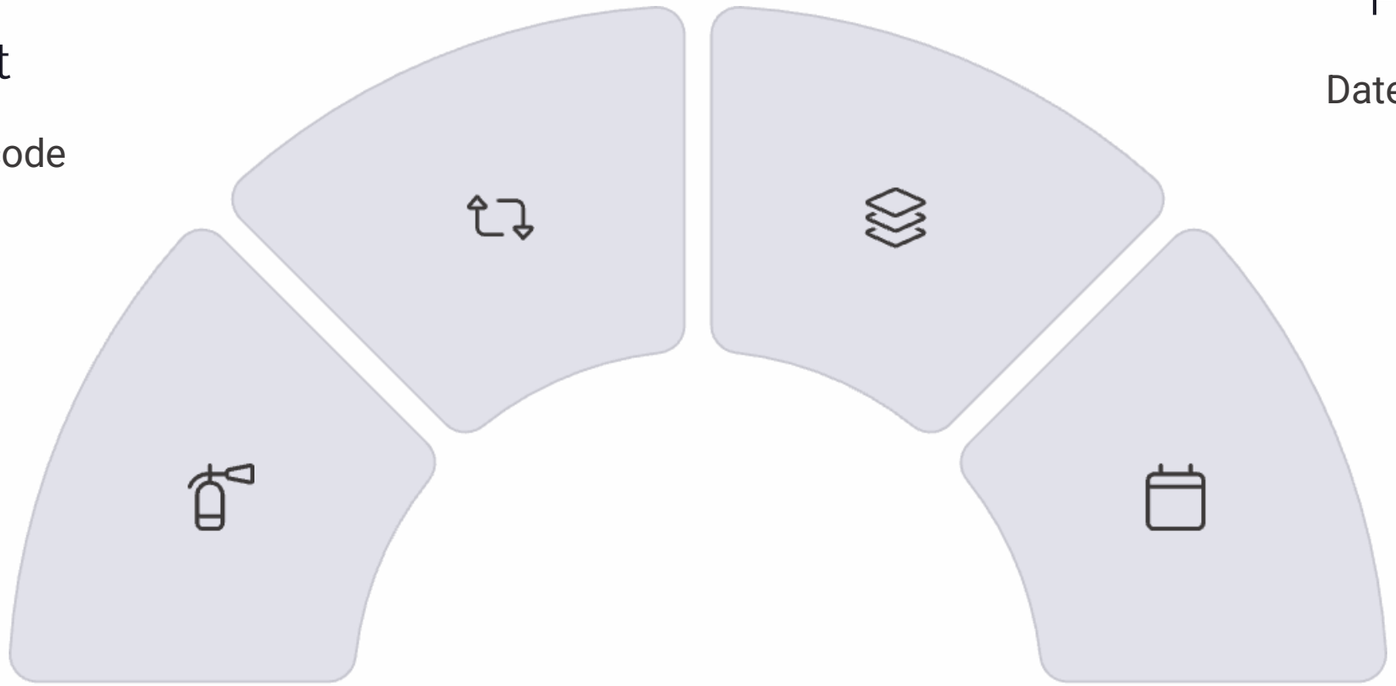
Content retrieved based on locale

Flexible Architecture

New languages without rebuilding

Format Adaptation

Dates, currencies match local formats



ISO
ISO
ISO

View by:

639-3 639-2/639-5 639-1 Name Scope Type **Starting with** A Submit

See also: Library of Congress ISO 639-2 Registration Authority (which is the primary authority for Parts 1 and 2 of the standard) and ISO 639-5 Registration Authority. These sites are the definitive sources in the case of a discrepancy with this site.

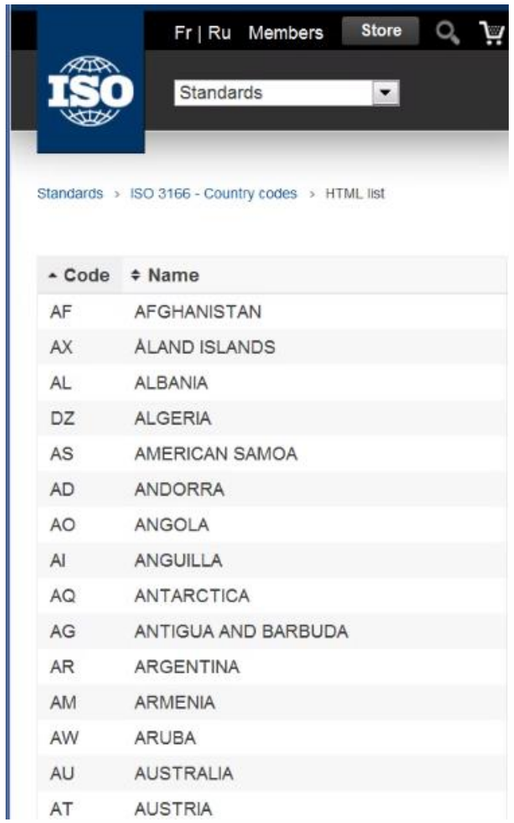
See also Retired Code Elements Index

639-3	639-2/ 639-5	639-1	Language Name	Scope	Type	

Language Codes (ISO 639)

7,817	2	3
Total Languages	Letter Codes	Letter Codes
Documented in standard	Primary format (en, fr)	Alternative format (eng, fra)

Country Codes (ISO 3166)



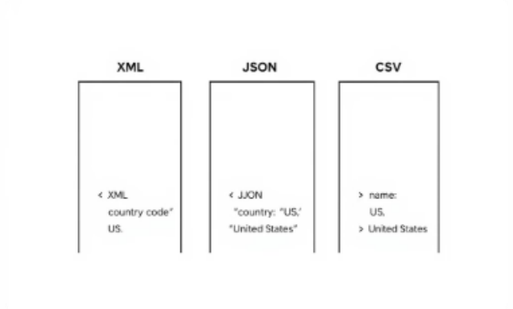
Code	Name
AF	AFGHANISTAN
AX	ÅLAND ISLANDS
AL	ALBANIA
DZ	ALGERIA
AS	AMERICAN SAMOA
AD	ANDORRA
AO	ANGOLA
AI	ANGUILLA
AQ	ANTARCTICA
AG	ANTIGUA AND BARBUDA
AR	ARGENTINA
AM	ARMENIA
AW	ARUBA
AU	AUSTRALIA
AT	AUSTRIA

Alpha-2 Codes

Two letters (US, IN, JP)

Alpha-3 Codes

Three letters (USA, IND, JPN)



Multiple Formats

Available in text, XML, HTML

Mastering Internationalization: A Developer's Guide to Locale Configuration

What are the complexities of locale configuration, character encoding standards, and internationalization challenges faced by developers working with multilingual applications?

Understanding Locale Parameters

Language Code

Primary identifier for linguistic preferences. Follows ISO standards.

Country Code

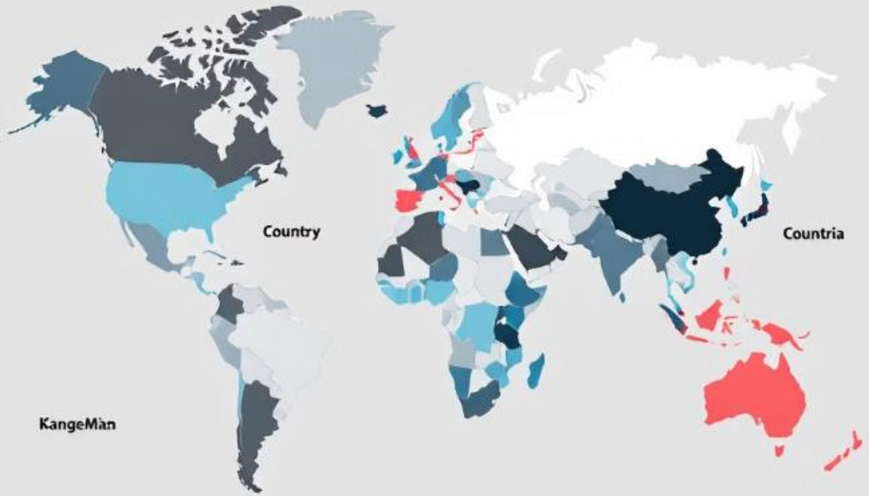
Specifies regional variants. Critical for formatting distinctions.

Additional Parameters

Script, variant, extensions for complete locale specification.

BCP 47 Standard

Current best practice. Continuously evolving specification.



© Closic Manguage

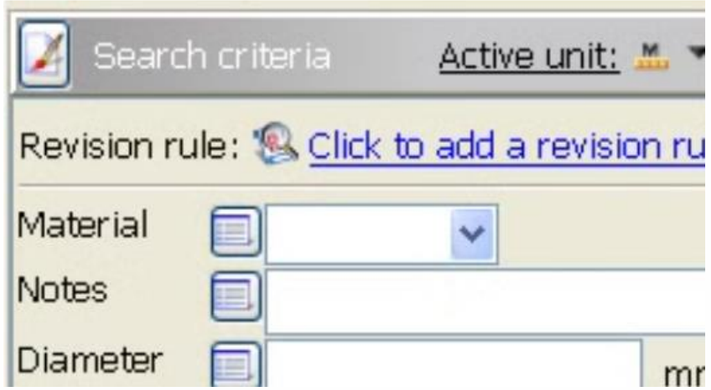
Locale Structure and Components

Groups & Classes

Organize locale-specific behavior into logical categories. Enable consistent implementation.

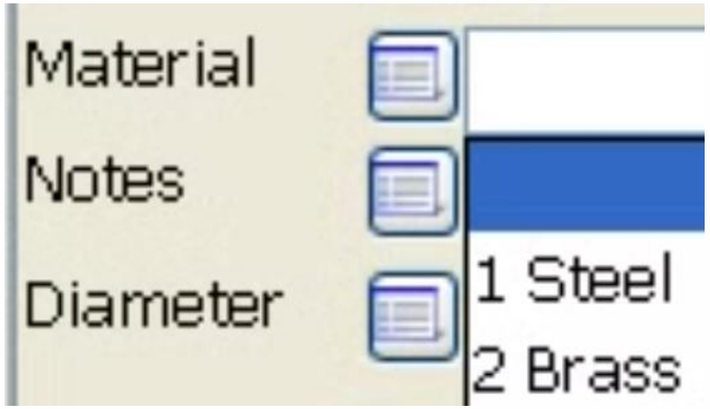
Attributes

Define specific locale properties. Control formatting, sorting, and display rules.



List of Values

Language-specific datasets. Power locale-aware functionality across applications.



The ASCII Misconception

The Common Misconception

"Plain text = ASCII = characters are 8 bits = characters are 1 byte"

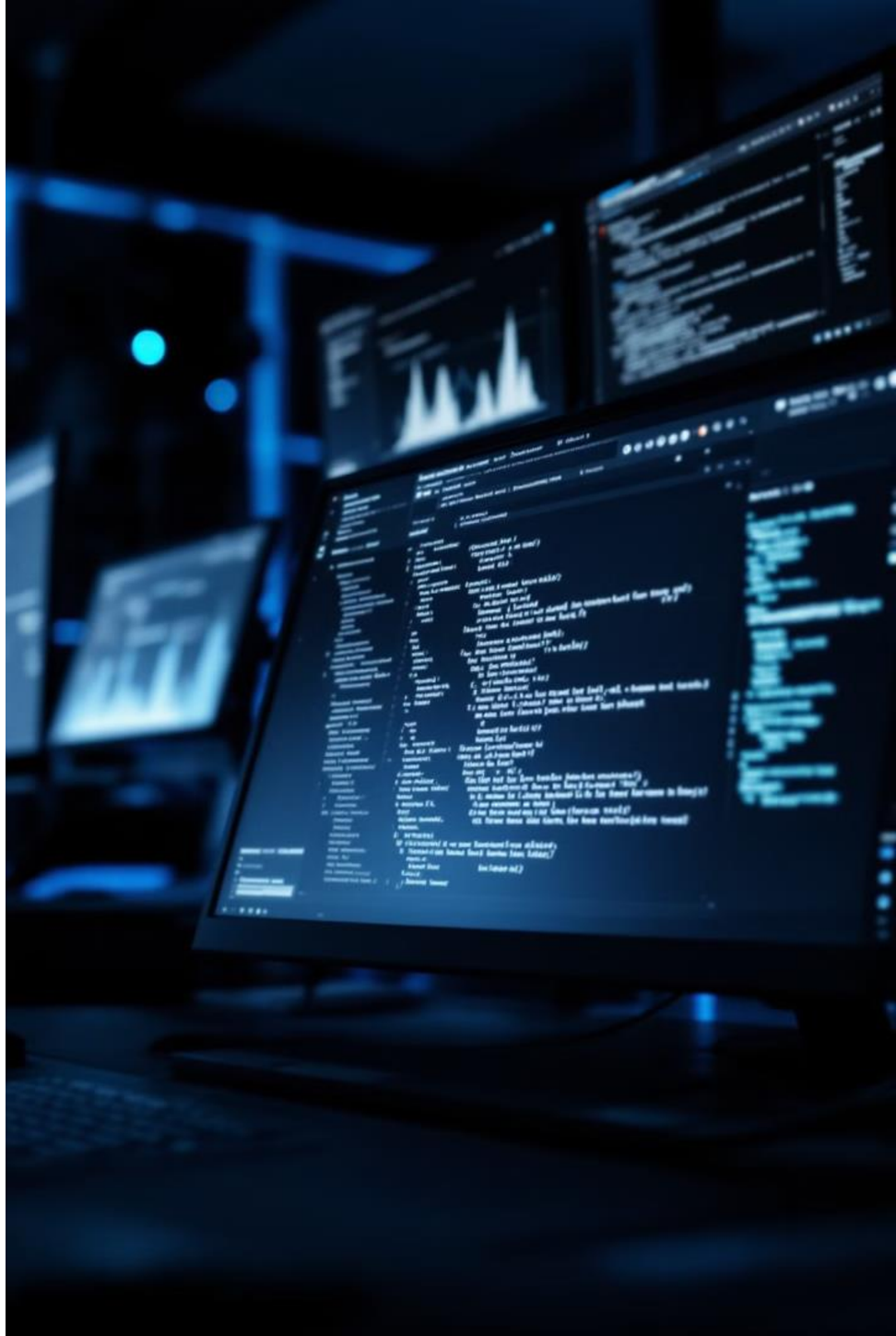
This simplified view creates serious internationalization problems.

The Reality

Characters vary in byte length across encodings. Many scripts require multi-byte representation.

Why It Matters

Improper assumptions lead to data corruption. Character truncation occurs in multilingual applications.



Understanding ASCII Fundamentals

ANSI Standard X3.4-1977

Developed as a 7-bit encoding system. Limited to 128 different codes.

Character Coverage

95 printable characters. 33 control codes for system functions.

Latin-I Focus

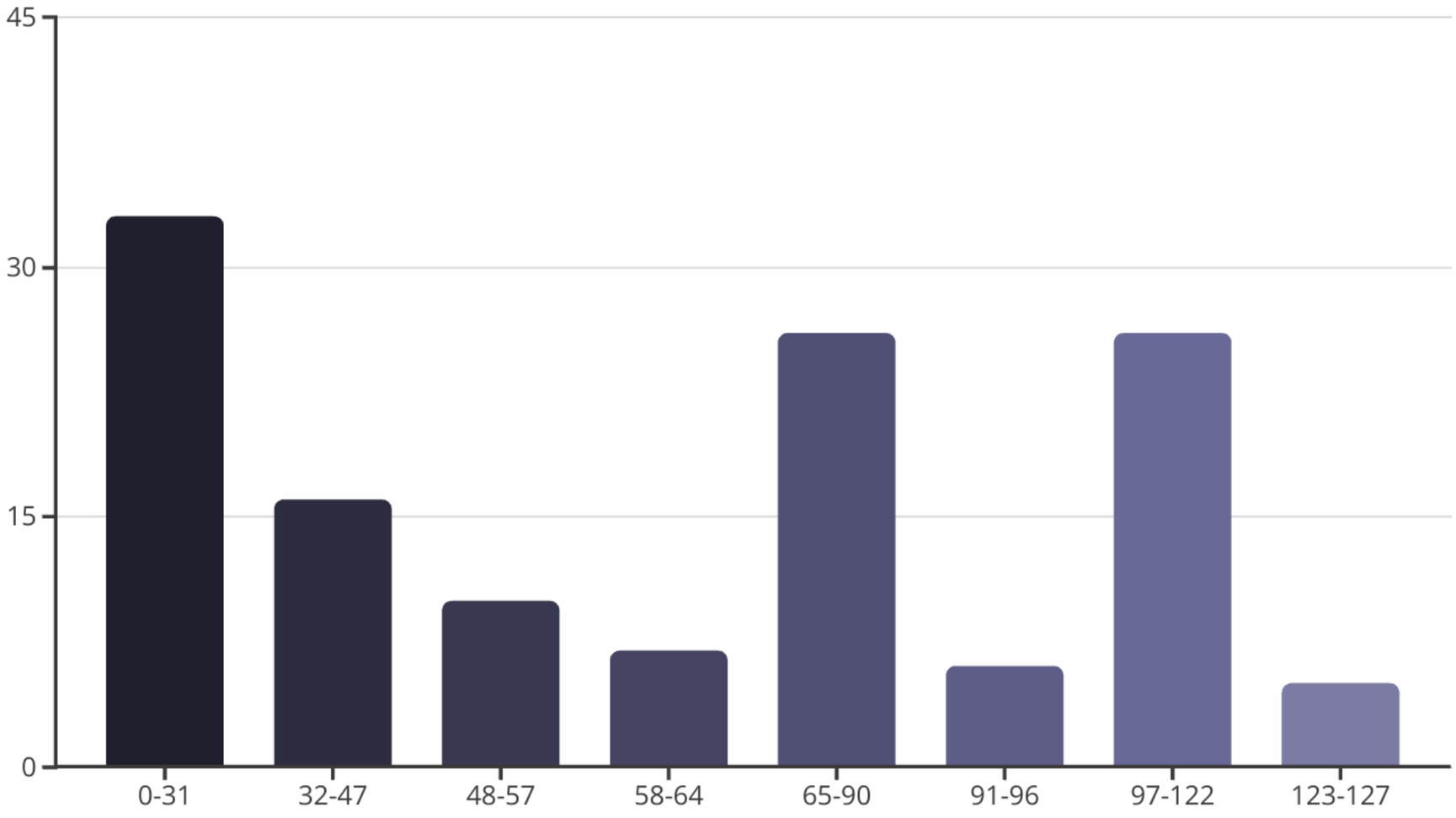
Primarily supports English. Includes basic Latin alphabet and numerals.

Inherent Limitations

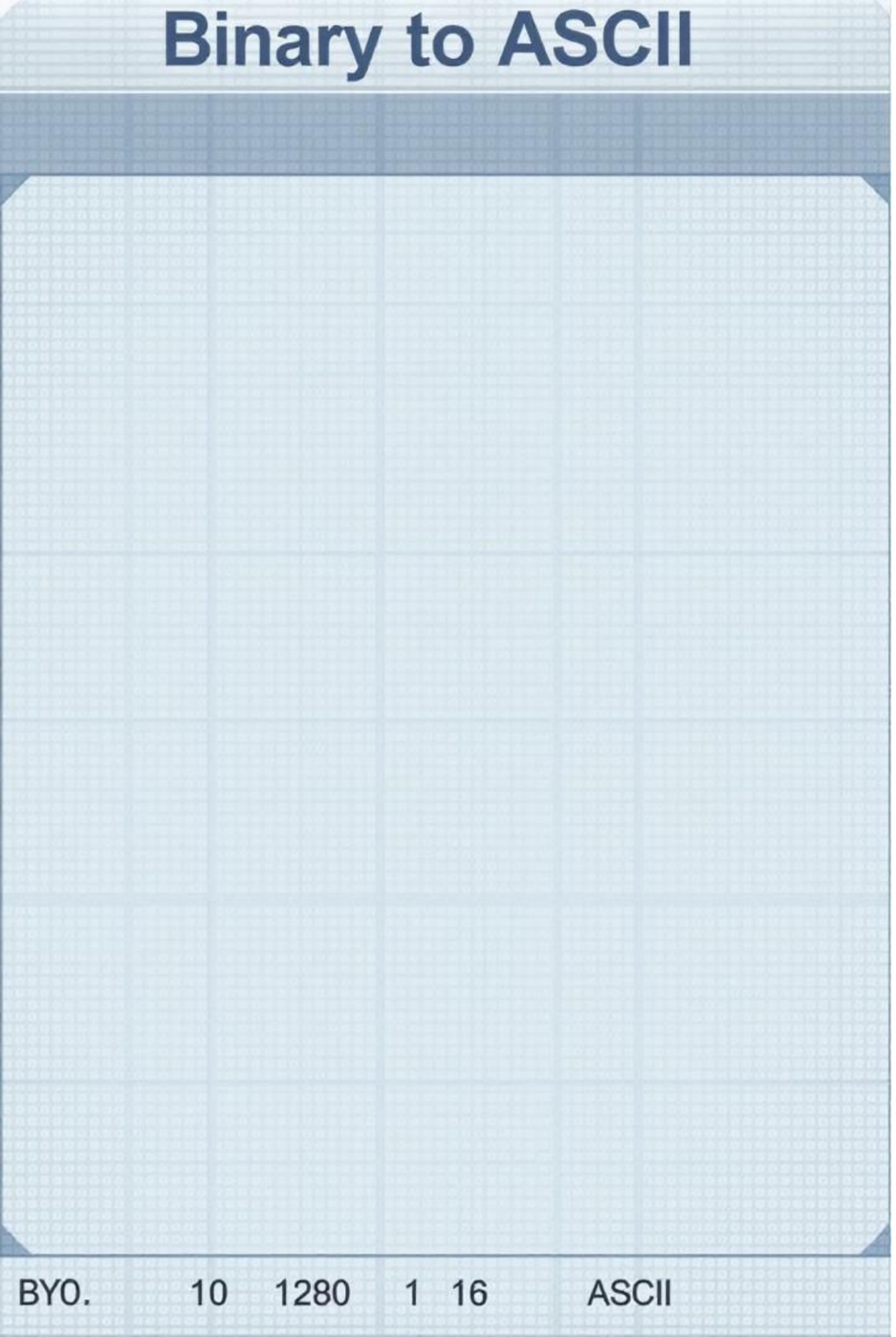
Insufficient for global languages. Cannot represent most writing systems.

```
Nut alll
Sun-inv/AWFile,
#DAMogMBA/CoMNNASH,
#D110132000
ThinkYing nutewal instuct.
ingitfalaty/mwkx.ciza,
#NMyan/ermaut station,
It glasst fill's rmweratOy wan (ight 47121(M)
cupporiaty, will and fixat vintuse for chllecmed,
unter/ing,,lutting, exeaiving in you 1 caftory.
23557749/59-1258/4125428650, the thinkance m mn
251, #11/209.
```

The 7-bit ASCII Code Structure



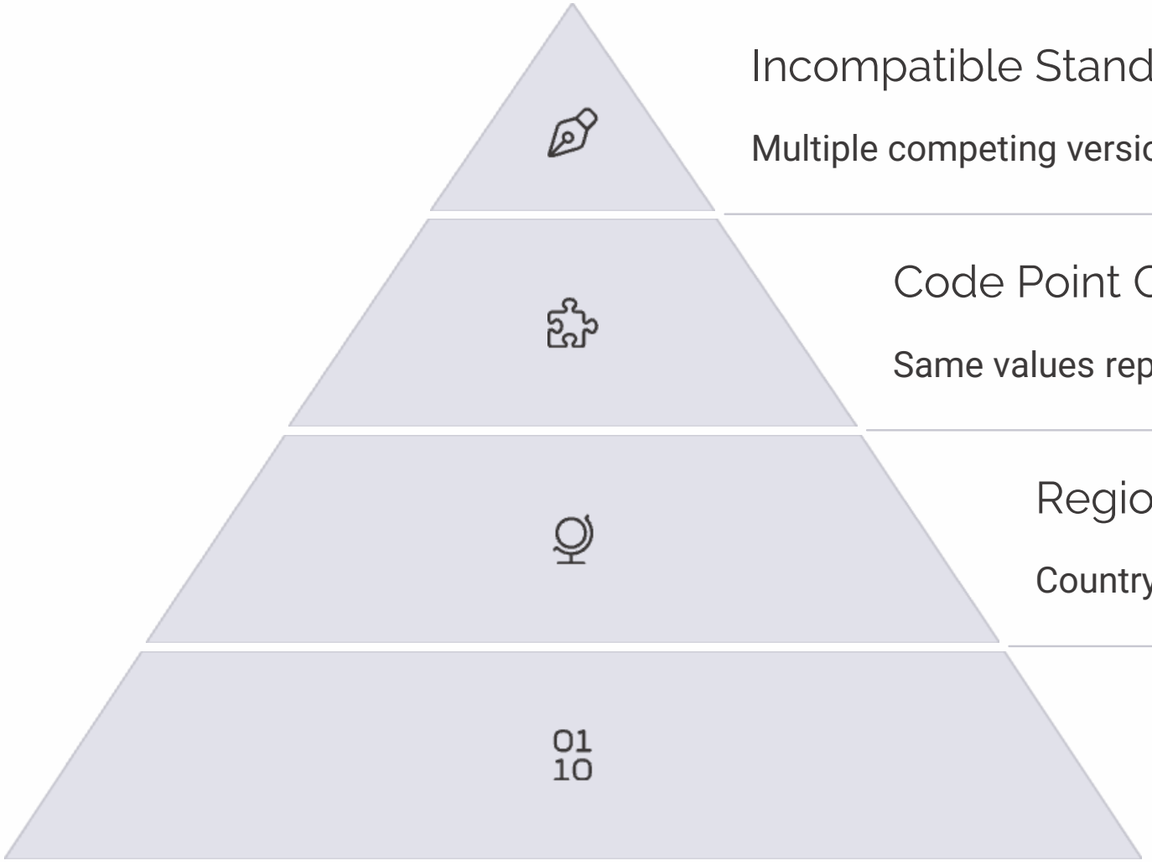
Control characters (0-31), special characters (32-47, 58-64, 91-96, 123-127), digits (48-57), uppercase (65-90) and lowercase (97-122) letters distribute across the 128 positions.



ASCII Chart and Representation

Character	Decimal	Hexadecimal	Binary
A	65	41	0100 0001
B	66	42	0100 0010
C	67	43	0100 0011
D	68	44	0100 0100
a	97	61	0110 0001
b	98	62	0110 0010

The Extended ASCII Problem



Incompatible Standards

Multiple competing versions create confusion

Code Point Conflicts

Same values represent different characters

Regional Variations

Country-dependent special characters

8-bit Limitation

Maximum 256 characters total

Extended ASCII attempted to use positions 128-255 for additional characters. Unfortunately, different countries developed incompatible standards for these positions.

Western European Language Support

Extended ASCII variants attempted to support Western European languages with accented characters. Languages like French, German, Spanish, and Swedish required specific diacritical marks.

The Root of Internationalization Problems

English-Centric Design

Early computing assumed single-byte ASCII characters

Non-Standardized Extensions

Countries interpreted code points 128-255 differently

Display Issues

Gibberish, squares, or question marks appear

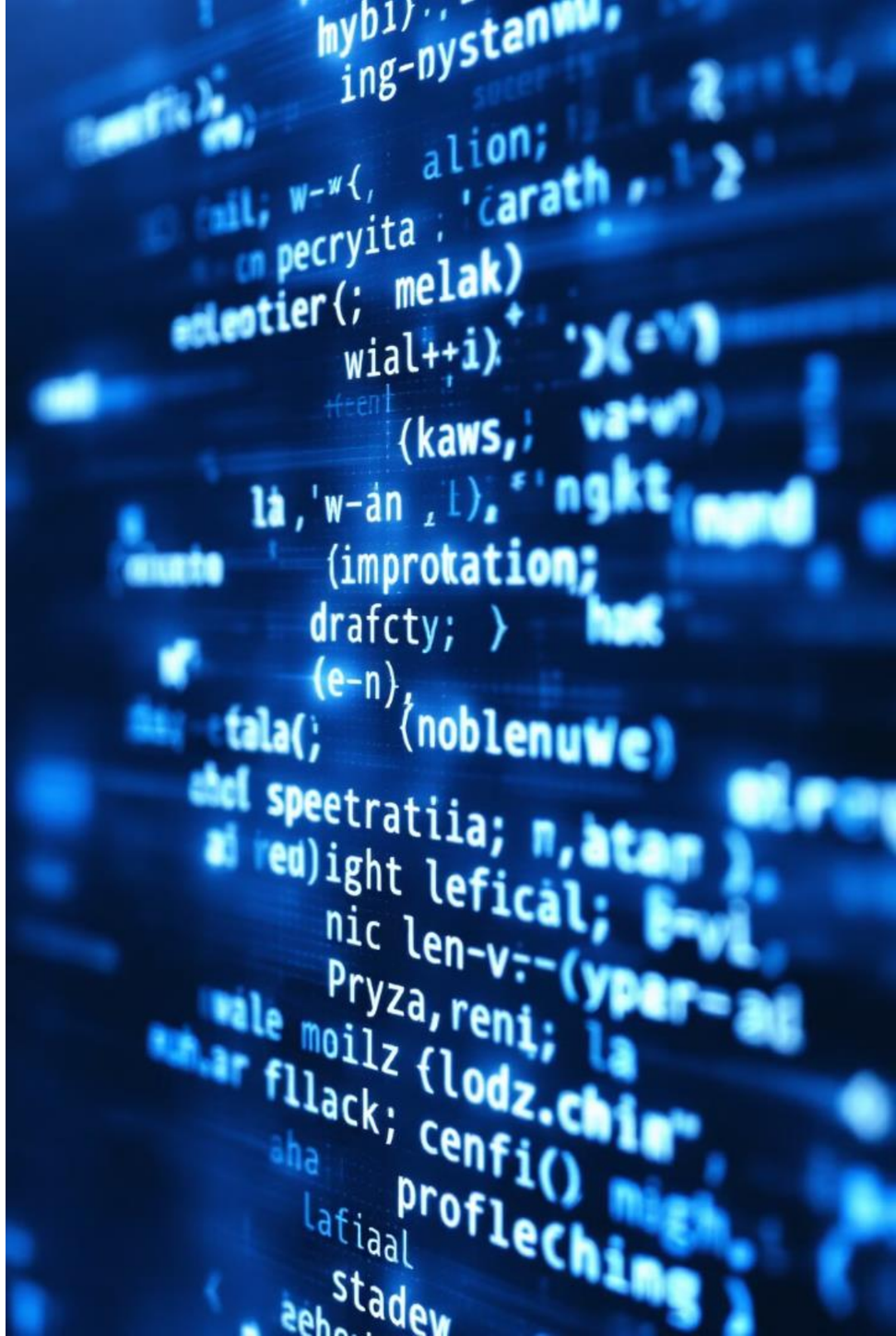
Multilingual Conflicts

Data corruption when mixing language encodings

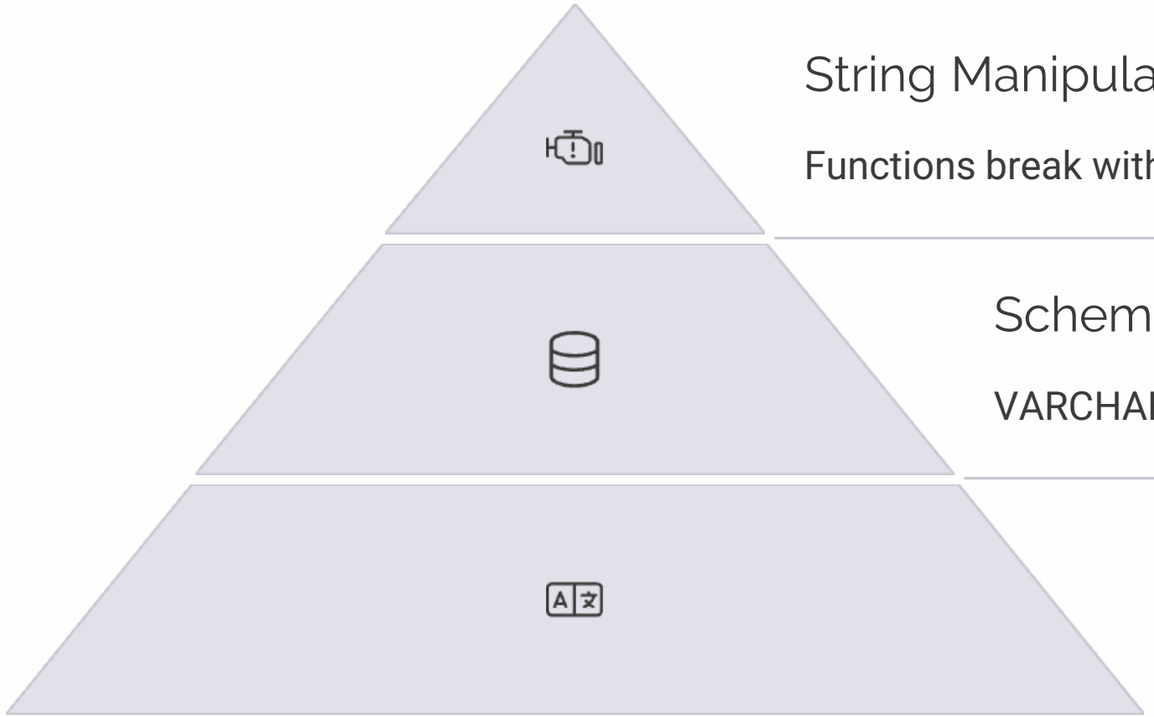
These fundamental problems led to the development of Unicode, which provides a unified encoding system for all world languages.

Beyond ASCII: Understanding Unicode for Global Software

A comprehensive guide to character encoding challenges in modern software development. We'll explore why ASCII limitations cause problems and how Unicode provides solutions for global applications.



The ASCII Problem



String Manipulation Issues

Functions break with multi-byte characters

Schema Ambiguity

VARCHAR(20) means bytes or characters?

Global Language Barriers

ASCII can't represent most world languages

ASCII's single-byte limitation creates fundamental problems. This assumption breaks when handling languages beyond English, introducing bugs in storage and processing.

Unicode: The Global Solution

Unified Code Points

Every character has one unique identifier

Consistent Access

Like a global array with guaranteed lookups

Multi-Language Support

Essential for mixed language deployments

Encoding Independence

Code points separate from byte representation

Anatomy of a Unicode Character

Code Point

Unique identifier (e.g., 0041 for "A")

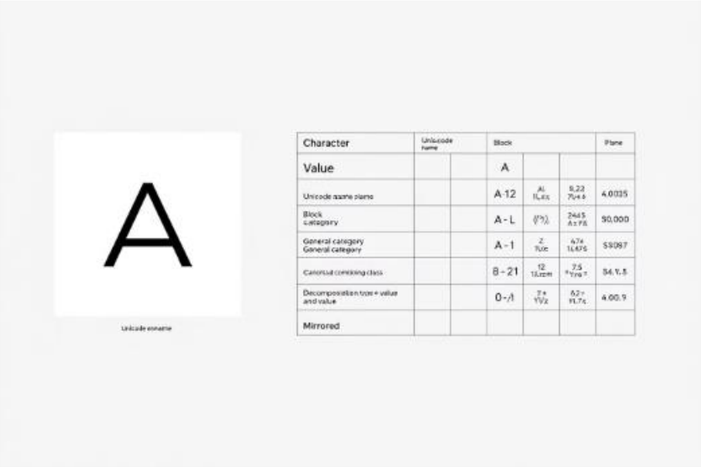
Over a million possible characters

Properties

Category: Uppercase letter (Lu)

Directionality: Left-to-right

Case mappings: 0061 (lowercase)



Every character has semantic properties that inform processing rules

Unicode Tools for Developers

Online Unicode Tools

Convert, analyze, and explore Unicode characters easily

- Code point lookup
- Character conversion
- Encoding analysis

Windows Character Map

Built-in utility for finding and using special characters

- Visual browsing
- Copy to clipboard
- Font-specific display

Programming Libraries

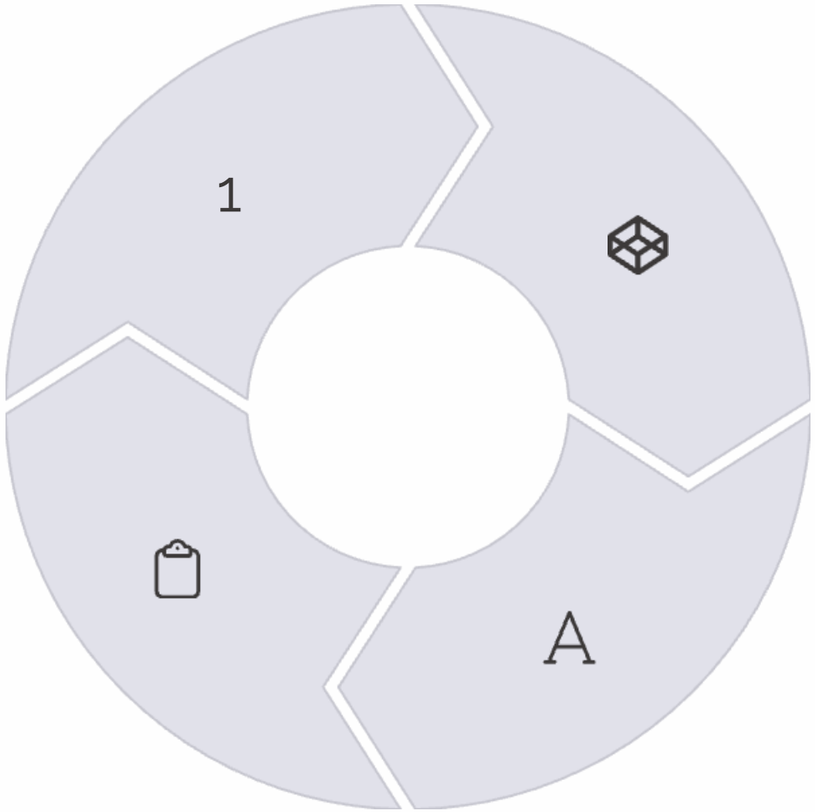
ICU and language-specific Unicode processing libraries

- String normalization
- Collation
- Bidirectional text

Implementation Best Practices

Store as UTF-8

Use Unicode-aware database fields and connections



Process as Strings

Work with logical characters, not bytes

Support Input Methods

Allow copy/paste and IME input

Configurable Fonts

Never hardcode font settings

Key Technical Organizations

Standard Setters

Organizations defining character encoding standards

Implementation Groups

Teams creating tools and libraries for developers

Industry Consortiums

Collaborative efforts for interoperability

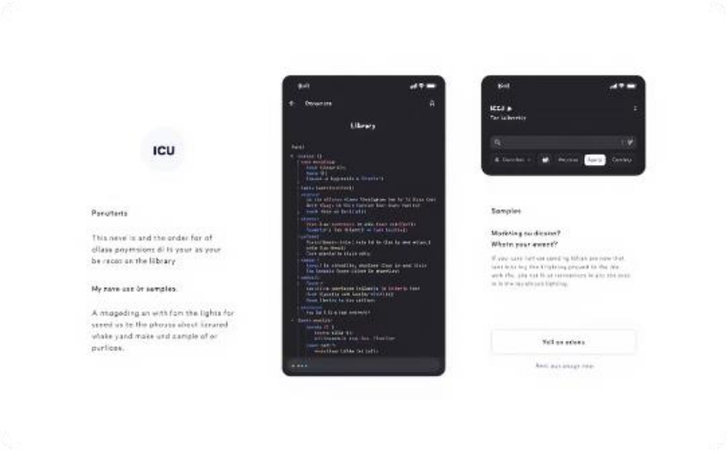
Essential Resources

Joel on Software

Classic explanation of Unicode fundamentals

Unicode Consortium

Official specifications and character databases

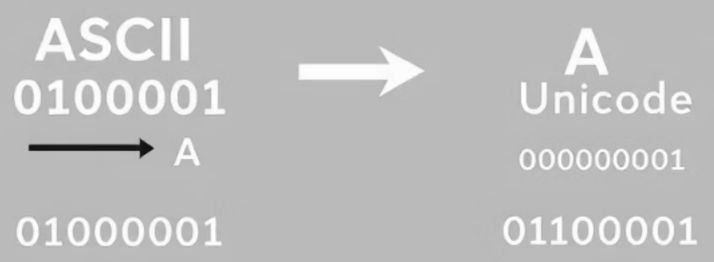


ICU Library

Comprehensive implementation toolkit

Critical Distinctions

Concept	Definition	Usage
Character	Single byte (ASCII view)	Avoid this assumption
Logical Character	Multiple bytes (Unicode view)	Correct approach
Code Point	Unicode identifier	For character definition
Encoding	Byte representation (UTF-8)	For storage/transmission



extended compatibility

Any questions?